

Privacy for ElectionGuard

Abstract—blank

Catalin: Change template

Index Terms—they exist

1. Introduction

Catalin: the story in my head:

- underlying primitives and their assumptions and sec properties.
- *non-repudiation* when complain is used. How does G_i know that (pki, pii) comes from G_i . **Catalin:** missing assumptions
- lay heavily on clarity for encryption schemes. DPP scheme is IND-CPA, but we prove this for the first time. Hybrid Enc is IND-CPA (find a reference or prove it)
- introduce known results for ENC-then-MAC - that it ensures NM-CPA(?). It could be called ENC-and-MAC. for sure it maintains IND-CPA. worth relating to authenticated ENC (AE), or make this whole section about AE and say how this ENC+MAC is one way of achieving this. **Catalin:** major flaw - we may need authentication of message coming from G_i to G_j . how does G_j know the ciphertext comes from G_i . It complains that the share is incorrect (after it decrypts a ciphertext); and then dispute between G_i and G_j makes unclear who is at fault; if Adv sent stuff to G_j .
- talk about threshold Enc - how does one achieve this from CPA scheme. Then, how DKG or VSS + CPA Enc makes th-ENC. Talk about the EG-Usenix paper results where EG-ELGamal is shown to be th-ENC. Here is would see some general definition of DKG, and some general and known theorems. **Catalin:** apparently one can do threshold proof using biased keys - see papers from Claudio orlandi - the EG-Usenix cites this paper.
- talk about ZKP proofs. known result Signed El-gamal and ElGamal + Schnorr gives NM-CPA. ... question for us - is there a general result that says CPA ENC+ PoK ZKP $\Rightarrow$ NM-CPA. **Catalin:** I think there was something on how to use signatures to make it IND-CCA, and schnorr is a signature.
- formalize EG, and compare with what the EG-Usenix paper has done.
- finally, threshold ballot privacy. **Catalin:** investigate major definition that look at threshold version of privacy. There is a paper at E-VOTE-ID on threshold receipt freeness. I think this is not perfect.
- other properties strong correctness and consistency in the context of threshold. how do they change.

- maybe, do we have a threshold definition of privacy - where we can show our th ballot privacy + th strong corr + th strong con makes this th privacy.
- finally x2. the proof for EG privacy
- if we think our contribution is light we can talk about verifiability for EG.

2. Preliminaries

2.1. Cryptographic primitives

Cyclic Group. We consider the cyclic group (p, q, g) defined by the generator g and primes p and q . As q is the order of the group the following property holds $p = r \cdot q + 1$ for some integer $r \in \mathbb{Z}_+^*$. For ease of expression, we introduce the algorithm

$$(p, q, g) \leftarrow \text{GSetup}(\lambda).$$

Hard Assumptions. DL, DDH **Catalin: continue**

Hash Functions. **Catalin: complete citation** We consider the hash functions as $H : \{0, 1\}^{256} \times \{0, 1\}^* \rightarrow \{0, 1\}^{256}$. When we require to reduce the hash to be in $\mathbb{Z}_q$, we consider $H_q(x; y) = H(x; y) \bmod q$ [?]. A hash function is *collision resistant* if no efficient adversary $\mathcal{A}$ can produce two distinct values y_1, y_2 such that $H(x, y_1) = H(x, y_2)$, for some x . ElectionGuard implements H using HMAC-SHA-256 [?].

2.2. Non-Interactive Proof Systems

We introduce non-interactive zero-knowledge proof systems $\text{PS} = (R, P, V)$ that consists of

- $\{0, 1\} \leftarrow R(w, st)$ a deterministic relation over a *statement* st and witness w . Sometimes, the statement may contain public parameters (e.g., the cyclic group used). It is also common for the relation to be NP, and in [?] ¹ it has been shown that this can be evaluated in polynomial time.
- $\pi \leftarrow P(w : st, [aux])$ is a probabilistic algorithm that produced a proof π for a statement st , given access to the witness w . One can transform this into an *signature of knowledge* [1] by including an auxiliary message $[aux]$.
- $\{0, 1\} \leftarrow V(st, \pi, [aux])$ is a deterministic algorithm that verifies the proof π for the statement st . *Completeness* ensures that the following holds with overwhelming probability $1 \leftarrow V(st, P(w : st, [aux]), [aux])$ for any valid relation $R(w, st)$.

The proof systems we consider are in the Random Oracle Model (ROM). We abstract away the ROM from the description by relying on [2] that states any result that was obtained using a blackbox reduction without ROM will also hold if the primitives and adversary are given ROM access.

We consider the *zero-knowledge property* (ZKP) [?] that ensures the proof leaks no information beyond the fact that the relation holds. This is formalized by having an efficient adversary $\mathcal{A}$ to distinguish between $\pi \leftarrow P(w : st, [aux])$ and $\pi \leftarrow \text{Sim}(st, [aux])$ for some simulator Sim .

Catalin: See if we need simulation sound extractability, proof of knowledge, soundness, special soundness, honest verified zero-knowledge. Catalin: maybe worth citing this <https://eprint.iacr.org/2018/164.pdf> to say any HVZK can lead to ZKP

1. Catalin: see <https://eprint.iacr.org/2024/1493.pdf>.

Instances of non-interactive zero-knowledge proof systems. We consider the following standard instantiations, formalized in Section B :

- SchZKP [3] for the knowledge of an exponent a without revealing it $R_3(a, A) = [A = g^a \bmod p]$. This is the case where $m = 1$ for ExpoZKP.
- ExpoZKP [4] for showing knowledge of multiple exponentiations $(a_1, \dots, a_m)$ where $\forall i \in [m]. A_i = g^{a_i} \bmod p$. That is, the relation is

$$R_3((a_1, \dots, a_m), (pp, A_1, \dots, A_m)) = [A_1 = g^{a_1} \bmod p \wedge \dots \wedge A_t = g^{a_t} \bmod p].$$

- ChPeZKP [5] for showing the validity of the encryption by demonstrating that the vote $v \in \{0, 1\}$ and knowledge of the randomness a used in encryption. The relation is defined as

$$R_7((v, a)(A, B)) = \left[\begin{array}{l} (A = g^a \wedge B = pk^{a+v}) \\ \vee (A = g^a \wedge B = pk^{a+(1-v)}) \end{array} \right].$$

Catalin: say something about their properties, soundness, zero-knowledge, etc.

2.3. Verifiable Secret Sharing

Catalin: See Traceable Variable Secret Sharing eprint 2025/318. What happens if decryption failed, currently it's ZKP. Catalin: This citation, doesn't actually make verify explicit.

We introduce the verifiable secret sharing [4], [6] $\text{VSS}(t, n) = (\text{Share}, \text{Rec}, \text{Vf})$ where $0 < t \leq n$ and

- $((sh_1, \dots, sh_n), \text{pub}) \leftarrow \text{Share}(pp, s)$ a probabilistic sharing procedure for secret s .
- $\{0, 1\} \leftarrow \text{Vf}(pp, \text{pub}, (j, sh_j))$ checks if a share is valid.
- $s_\perp \leftarrow \text{Rec}(pp, J, (sh_j)_{j \in J})$ recovers the secret s if $t \leq |J|$, and $\perp$ otherwise.
- $S_\perp \leftarrow \text{RecE}(pp, J, \{SH_j\}_{j \in J})$ recovers the secret s in the exponent $S = g^s$ for all $SH_j = g^{sh_j}$. Pedersen [7] has shown for Shamir secret sharing [8] (the underlying scheme used by Feldman [9] and ElectionGuard) that

$$\text{RecE}(pp, J, \{SH_j\}_{j \in J}) = g^{\text{Rec}(pp, J, (sh_j)_{j \in J})},$$

while still maintaining the same privacy guarantees as the initial reconstruction. We abuse notation $(S_\perp, \{L_j \mid j \in J\}) \leftarrow \text{RecE}$ to also output publicly available values L_j for each party (e.g., the Lagrange coefficients).

The sharing phase is optimally composed [6] of (1) the dealer distributing the shares (i.e. by sending the output of Share) and (2) the other participants verifying their shares (i.e., by calling Vf) and raising and resolving the complains. This is often modeled in the context of having a communication channel that is either an encrypted and authenticated channels [6] or uses encrypted-and-signed messages [4]. *Correctness* ensures that any honest run of $\text{Rec}(pp, J, \text{Share}(pp, s))$ for $t \leq |J|$ will produce the secret s , and valid share verifies $1 \leftarrow \text{Vf}(\text{Share}(s))$. *Secrecy* states that if the dealer is honest, then during the sharing phase the adversary learns no information about the secret

that is shared (as long as the adversary corrupts/controls at most $(t - 1)$ parties).

Security looks at the context of a dishonest dealer during the sharing phase and aiming to convince all honest parties that they hold valid and consistent shares. Depending on size of the honest parties, one has either *honest majority* - that is the threshold satisfies $0 < t \leq n/2$, or *dishonest majority* with $0 < t \leq n$. A special type of security is *with aborts* where the guarantee of having a valid and consistent share is for honest parties that didn't abort, and nothing can be said about the parties that aborted.

Remark 1. Secrecy aims to guarantee no leakage of information about the secret from shares (assuming the dealer is honest), while security will guarantee recovery of the secret even in the context of dishonest dealer. Both properties rely on the fact that the adversary only controls at most $(t - 1)$ parties.

Catalin: talk about disputes and complains.

Instantiations of Verifiable Secret Sharing. ElectionGuard relies on variants of Feldman's verifiable secret sharing schemes [9]:

- VSS-v2.0.0: Feldman with multiple SchZKP. The dealer publishes commitments to the coefficients of the polynomial g^{a_i} for all i , together with individual Schnorr Proofs $\pi_i \leftarrow \text{SchZKP}(a_i : \text{pp}, g^{a_i}, j)$ that it knows the exponent (i.e., the coefficient). This is formalized in Figure 1.
- VSS-v2.1.0 [4]: Feldman with ExpoZKP. This is similar to VSS-v2.0.0 except the dealer publishing a single ExpoZKP proof for all coefficient, instead of individual ones.

Theorem 1 ([4]).²

VSS-v2.0.0 and VSS-v2.1.0 satisfy correctness, secrecy, security with honest majority and security with dishonest majority and aborts, assuming the communication between the parties is confidential and authenticated.

Additive Homomorphic Secret Sharing [10].

ElectionGuard relies on their parties combining shares from multiple secret sharings. In essence, each party will have a combined share as the sum of all the shares that that party received $sh_j = \sum_{i=1}^n sh_{i,j}$ for $(\dots, sh_j, \dots) \leftarrow \text{Share}(\text{pp}, (\sum_{i=1}^n s_i))$ and $(\dots, sh_{i,j}, \dots) \leftarrow \text{Share}(\text{pp}, s_i)$. We abstract this as

$$\text{Share}(\text{pp}, (\sum_{i=1}^n s_i)) = \sum_{i=1}^n \text{Share}(\text{pp}, s_i).$$

It is straightforward to see that the combine secret sharing scheme maintains the same properties as the underlying sharing schemes.

2.4. Multi-Prover Zero-knowledge Proof Systems

They were introduced by Ben-Or et al. [11] for the interactive case, as a mechanism to allow multiple provers

² [4] analyses only VSS-v2.1.0, but it's straightforward to see that VSS-v2.1.0 and VSS-v2.0.0 are equivalent from a security perspective.

to collaboratively convince a verifier of a single statement. Cui et al [12] have shown that one can apply a version of the Fiat-Shamir transformation to make them non-interactive.

Catalin: check if this is novel, or already proposed by others **Catalin:** investigate the notion of aggregate zero-knowledge for multi-provers and DL relation.

In Figure 2 we introduce a multi-prover zero-knowledge MPZKP where each prover has independently chosen witnesses $(i, \text{lesk}_i, \text{lepk}_i)$ and together they prove that they know the exponent $\text{lesk} = \sum_{i \in I} \text{lesk}_i$. More precisely, the relation is defined as

$$\begin{aligned} R((\{(i, \text{lesk}_i) \mid i \in I\}), (\text{pp}, A, I, \{(i, A_i, \text{lepk}_i) \mid i \in I\})) \\ = (\prod_{i \in I} A_i = A^{\sum_{i \in I} \text{lesk}_i}). \end{aligned}$$

Secret Sharing and MPZKP. One can apply MPZKP in the context where the witnesses lesk_i are not independent, and have been build using secret sharing. Consider the secret esk has been shared into secret shares $(\dots, \text{esk}_i, \dots, \text{pub}) \leftarrow \text{Share}(\text{pp}, \text{esk})$. The system then becomes a proof of knowledge of the reconstruction, if the reconstruction can be rewritten as a linear equation of the secret shares

$$\text{Rec}(\text{pp}, I, (\text{esk}_i)_{i \in I}) = (\sum_{i \in I} \text{esk}_i \cdot L_i, \text{ for some } L_i) \bmod q.$$

In the case of ElectionGuard (based on using VSS-v2.0.0 or VSS-v2.1.0), this is achieved by $\text{lesk}_i = \text{esk}_i \cdot L_i$ for the Lagrange coefficient $L_i = \prod_{j \in I \setminus \{i\}} (-j/(i-j)) \bmod q$. Additionally, they rely on the additive homomphoc secret sharing property to merge multiple sharing procedures such that $\text{esk}_i = \sum_{j \in J} sh_{i,j}$ with these shares being produced by each party using $(\dots, sh_{i,j}, \dots) \leftarrow \text{Share}(\text{pp}, s_i)$.

One can also consider secret sharing schemes based on the Chinese Remainder Theorem [13], [14], where $\text{lesk}_i = \text{esk}_i \cdot M_i$ for $M_i = (M/m_i) \cdot ((M/m_i)^{-1} \bmod m_i) \bmod M$ given the fix product M of all co-primes m_i for $i \in I$. To account for the different style of sharing $q = m_0$ the space where the secret is chosen from. This would be similarly to the proof of multi-exponentiation by Yuen et al [15].

As the differences between VSS-v2.0.0 or VSS-v2.1.0 are only with respect to the type of zero-knowledge used in the sharing procedure, the following result holds for both, even if [16] only showed it for the case of VSS-v2.1.0.

Theorem 2. [16] MPZKP with VSS-v2.0.0 or VSS-v2.1.0 is correct, sound and zero-knowledge.

Remark 2. Benaloh et al [16] link their zero-knowledge design to the Sparkle threshold signature [?], however they don't require to evaluate any threshold condition in the proof system itself. Once they decide on the set of parties I involved in the decryption and proof, it becomes a simple multi-prover proof for a linear equation. Therefore, any other multi-prover proof could be used if more efficient constructions would be designed in the future. **Catalin:** See if no other construction exist.

Share (pp, s, [aux])	Vf (pp, pub, (j, sh _j), [aux])	Rec (pp, J, (sh _j) _{j∈J} , [aux])
1 : ((p, q, g), t, n) ← pp 2 : a ₀ ← s 3 : A ₀ ← g ^{a₀} 4 : for j ∈ {1, ..., t-1} do 5 : a _j ← \$Z _q 6 : A _j ← g ^{a_j} mod p 7 : π _i ← P ₃ (a _j : pp, A _j , [aux, j]) 8 : P(x) = a _{t-1} x ^{t-1} + ... + a ₁ x + a ₀ 9 : for ℓ ∈ {1, ..., n} do 10 : sh _ℓ ← P(ℓ) 11 : π ← (π ₀ , ..., π _{t-1}) 12 : pub ← (A ₀ , ..., A _{t-1} , π) 13 : return ((sh ₁ , ..., sh _n), pub)	1 : ((p, q, g), t, n) ← pp 2 : (A ₀ , ..., A _{t-1} , π) ← pub 3 : S _j ← ∏ _{ℓ=0} ^t A _ℓ ^{j^ℓ} mod p 4 : e _t ← g ^{sh_j} = S _j 5 : e ← PubVf(pp, pub, [aux]) 6 : return e ∧ e _t	1 : ((p, q, g), t, n) ← pp 2 : if ¬PubVf(pp, pub, [aux]) return ⊥ 3 : for j ∈ J do 4 : L _j ← ∏ _{i∈J\{j}} (-i/(j-i)) mod q 5 : s ← ∑ _{j∈J} sh _j · L _j mod q 6 : return s
	PubVf (pp, pub, [aux])	RecE (pp, J, (SH _j) _{j∈J} , [aux])
	1 : ((p, q, g), t, n) ← pp 2 : (A ₀ , ..., A _{t-1} , π) ← pub 3 : (π ₀ , ..., π _{t-1}) ← π 4 : for j ∈ {0, ..., t-1} do 5 : e _i ← Vf(pp, A _j , π _j , [aux, j]) 6 : e ← e ₀ ∧ ... ∧ e _{t-1} 7 : return e	1 : ((p, q, g), t, n) ← pp 2 : if ¬PubVf(pp, pub, [aux]) return ⊥ 3 : for j ∈ J do 4 : L _j ← ∏ _{i∈J\{j}} (-i/(j-i)) mod q 5 : S ← ∏ _{j∈J} SH _j ^{L_j} mod q 6 : return (S, {L _j j ∈ J})

Figure 1. Verifiable Secret Sharing [4] VSS-v2.0.0 with SchZKP = (R, P, V).

P (({lesk _i i ∈ I} : pp, {(A _i , lepk _i) i ∈ I}, I, (A, B), [aux])
Party (i, lesk _i , lepk _i , A _i) does the following
/ commitment phase
1 : r _i ← \$Z _q 2 : U _i ← g ^{r_i} mod p, V _i ← A ^{r_i} mod p 3 : PL ← (i, U _i , V _i) 4 : wait and collect ∀j ≠ i. (ℓ, U _j , V _j) 5 : U ← ∏ _{i∈I} U _i mod p, V ← ∏ _{i∈I} V _i mod p
/ challenge and response phase
6 : AS ← ∏ _{i∈I} A _i mod p 7 : c ← H(H _E ; A, B, U, V, epk, AS, [aux]) 8 : z _i ← (r _i - c · lesk _i) mod q 9 : PL ← (i, z _i) 10 : wait and collect ∀j ≠ i. (j, z _j)
/ validate responses
11 : ∀j ≠ i. U _j ['] ← g ^{z_j} · lepk _j ^c 12 : ∀j ≠ i. V _j ['] ← A ^{z_j} · A _j ^c 13 : ∀j ≠ i. if (U _j , V _j) ≠ (U _j ['] , V _j [']) then 14 : evidence ← (j, U _j , V _j , U _j ['] , V _j [']) 15 : O ← O ∪ {Dispute(i, j, evidence)}
/ the proof
16 : z ← ∑ _{i∈I} z _i mod q 17 : π ← (c, z) 18 : return π

Figure 2. Multi-Prover Zero-Knowledge Proof System MPZKP.

KGen₁ ((p, q, g))
1 : sk ← \$Z _q [*] ; pk ← g ^{sk} mod p 2 : return (sk, pk)
Enc₂ ((p, q, g), pk, m, aux ₁ , aux ₂)
1 : r ← \$Z _q 2 : c ₀ ← g ^r mod p 3 : β ← pk ^r mod p 4 : k ← H(H _P ; aux ₁ , pk, c ₀ , β) 5 : k ₀ ← H(k; 0x01, aux ₂) 6 : k ₁ ← H(k; 0x02, aux ₂) 7 : c ₁ ← m ⊕ k ₁ 8 : c ₂ ← H(k ₀ , c ₀ , c ₁) 9 : return ((c ₀ , c ₁ , c ₂), r)
Dec₂ ((p, q, g), sk, (c ₀ , c ₁ , c ₂), aux ₁ , aux ₂)
1 : β ← c ₀ ^{sk} mod p 2 : k ← H(H _P ; aux ₁ , pk, c ₀ , β) 3 : k ₀ ← H(k; 0x01, aux ₂) 4 : k ₁ ← H(k; 0x02, aux ₂) 5 : m ← c ₁ ⊕ k ₁ 6 : c ₂ ['] ← H(k ₀ , c ₀ , c ₁) 7 : if c ₂ ≠ c ₂ ['] then m ← ⊥ 8 : return m

Figure 3. ElectionGuard Variant of DHIES.

2.5. Public Key Encryption

We introduce the algorithms for the public-key encryption PKE = (KGen, Enc, Dec) with

- (pk, sk) ← KGen(λ) takes the security parameter λ and produces a public-secret key pair.
- c ← Enc(pk, m) returns the ciphertext c that encrypts the message m using the public key

pk. When we need the randomness we use c ← Enc(pk, m; r) or (c, r) ← Enc(pk, m).

- m_⊥ ← Dec(sk, c) returns either a valid message m or an error symbol ⊥. *Correctness* ensures with overwhelming probability that m ← Dec(sk, Enc(pk, m)) for any (pk, sk) ← KGen(λ).

We extend the definition to *homomorphic public key encryption* by including the algorithm

- c ← Add(c₁, c₂) where the ciphertext c is

the encryption of $(m_1 + m_2)$ for $(c_1, c_2) \leftarrow (\text{Enc}(\text{pk}, m_1), \text{Enc}(\text{pk}, m_2))$.

Security. All PKE schemes satisfy *indistinguishability under chosen plaintext attack* (IND-CPA) where the adversary is incapable of distinguishing between encryption of two messages, in the context of them choosing the messages and having no access to decryption oracle. Stronger notions of security do consider decryption oracle access for the adversary, and they are either limited to only one decryption query for a list of ciphertexts (called, *indistinguishability under chosen-ciphertext attack with one parallel decryption query* (IND-1-CCA) [17], [18]³) or many decryption queries (called, *indistinguishability under chosen-ciphertext attack* (IND-CCA2) [19]). Based on [19] and [17] we have the following relations between security properties:

$$\text{IND-CPA} \Leftarrow \text{IND-1-CCA} \Leftarrow \text{IND-CCA2}$$

Threshold PKE. A particular type of PKE are *threshold public-key encryptions* [20] $\text{TPKE}(t, n) = (\text{KGen}, \text{Enc}, \text{Dec}, \text{Combine})$ where the single secret key is split into secret shares during KGen and a threshold number of them are required to decrypt (see Appendix A.1). *Correctness* in this case ensures with overwhelming probability that $m \leftarrow \text{Combine}(\text{pkc}, I, \{d_i\}_{i \in I})$ over a set of $t \leq |I|$ partial decryption values $d_i \leftarrow \text{Dec}(\text{sk}_i, c)$ with $i \in I$, given the ciphertext $c \leftarrow \text{Enc}(\text{pk}, m)$ and keys $(\text{pk}, \text{pkc}, \text{sk}_1, \dots, \text{sk}_n) \leftarrow \text{KGen}(\lambda, t, n)$. IND-CPA in this context gives to the adversary the ability to corrupt up to $(t - 1)$ secret share keys.

A property specific to some TKPE is *robustness* where parties cannot lie about their partial decryption, mostly by providing a proofs of correct partial decryptions that anyone can check. This is designed and formalized as each party producing their own proof. ElectionGuard v2.0.0 and v2.1.0 rely on a multi-Prover ZKP where all parties act together to produce the evidence. **Catalin: This paper also talks about accountability and traitor tracing. See if relevant.**

DPKE: Devillez et al. encryption scheme [21]. This is a more efficient variant of ElGamal where the encryption step is $(g^r \bmod p, \text{pk}^{r+m} \bmod p)$ and decryption is done as $\log_{\text{pk}}(\text{pk}^{r+m}/(g^r)^{\text{sk}})$. The scheme DPKE = $(\text{KGen}_1, \text{Enc}_1, \text{Dec}_1, \text{Add}_1)$ has not been shown to be IND-CPA or claimed to be in the initial paper [21]. However, it's assumed to be IND-CPA in [16] when used with a distributed key generation to design a threshold encryption. For completeness we show that this construction satisfies IND-CPA.

Lemma 3. DPKE satisfies IND-CPA under the DDH assumption.

Proof. Similar to ElGamal this reduces to DDH. More precisely, the DDH adversary $\mathcal{B}(g^x, g^y, Z)$ return $(\beta = \beta')$ where the IND-CPA adversary produces $\beta' \leftarrow \mathcal{A}(g^y, (g^x, Z \cdot (g^y)^{m_\beta}))$ for some random choice $\beta \leftarrow \{0, 1\}$. $\square$

3. In [18] this was called poly-IND-1-CCA, and used to show Helios is ballot private. This is equivalent to NM-CPA [19].

PKEE: ElectionGuard variant of DHIES [22]. The scheme EPKE = $(\text{KGen}_2, \text{Enc}_2, \text{Dec}_2)$, described in Figure 3 is similar to DHIES, with the only difference being that in DHIES the keys are computed slightly different: $(k_0, k_1) \leftarrow k$. DHIES has been shown to be IND-CCA2 in [22] based on the **Catalin: oracle Diffie-Hellman assumption** and it is reasonable to expect that the EPKE also satisfies IND-CCA2⁴. Additionally, one can rely on the more general result from the Encrypt-then-MAC methodology [23] that says EPKE is IND-CCA2 and IND-CTXT if the underlying encryption scheme is IND-CPA and the MAC is sUF-CMA.

Remark 3 (Lack of authentication). This scheme satisfies *confidentiality* (due to IND-CCA2) and *integrity* (due to IND-CTXT). It is straightforward to see that it does not provide *authentication* or *non-repudiation*, as there is no mechanism to identify who created the ciphertexts.

We will show this is problematic as anyone can spam the setup phase without accountability. Moreover, they can create unsolvable disputes by impersonating a guardian in the second part of setup and sending fake shares. Any guardian receiving fake shares will report this but there is no method to determine if an external party send the message or a malicious guardian.

One method to include non-repudiation (via UF-CMA) is by using *Sign-then-Encrypt* methodology [24] while preserving the IND-CCA2 property of the encryption scheme.

Catalin: talk about what th enc is, what are the security properties, and what has been shown for EG-PKE

Catalin: state that ind-cpa has been shown, but not ind-1-cca and that is essential to voting. You can rely on (??) on standard convention to make ind-cpa to ind-cca via ZKP, then it needs to show that the ind-cca enc is also threshold ind-cpa.

4. Notice that the difference is between $(k_0, k_1) \leftarrow H(x)$ versus $k_0 \leftarrow H(0, H(x))$ and $k_1 \leftarrow H(1, H(x))$.

3. ElectionGuard v2.0.0

Entities and their role.

- **Election Authority** EA sets the election manifest that contains for example, the list of candidates, list of eligible voters, number of guardians and threshold.
- **Guardians** G own a share of the secret key, such that collectively they provide the election result and ensure correctness of the process while maintaining privacy. We consider a threshold model (t, n) where at least t out of the total n Guardians are needed.
- **Voters** id cast their vote.
- Election Record PL that stores publicly verifiable information.

Parameter generation and verification. Before the election starts, the following public parameters are agreed $pp = (\text{ver}, (p, q, g), \text{manifest}, t, n)$, where $\text{ver} = \text{"v2.0.0"}$, and $(p, q, g) \leftarrow \text{\$ GSetup}$ are a cyclic group and (t, n) the threshold and total number of guardians. The Election Authority defines $(H_P, H_M, H_B) \leftarrow \text{GenHash}(pp)$, and anyone can verify by recomputing the values.

GenHash(pp) with $pp = (\text{ver}, (p, q, g), \text{manifest}, t, n)$		
1 :	$H_P \leftarrow H(\text{ver}; 0x00, (p, q, g))$	/ Parameter Base Hash
2 :	$H_M \leftarrow H(H_P; 0x01, \text{manifest})$	/ Election Manifest Hash
3 :	$H_B \leftarrow H(H_P; 0x02, H_M, n, t)$	/ Election Base Hash
4 :	return (H_P, H_M, H_B)	

3.1. Primitives

We consider the public key encryption schemes defined in Section 2.1: $\text{DPKE} = (\text{KGen}_1, \text{Enc}_1, \text{Dec}_1, \text{Add}_1)$, $\text{EPKE} = (\text{KGen}_2, \text{Enc}_2, \text{Dec}_2)$, the zero-knowledge proof system $\text{SchZKP} = (P_3, V_3, R_3)$, verifiable secret sharing scheme $\text{CLVSS}_4 = (\text{Share}_4, \text{Rec}_4, \text{Vf}_4)$, the zero-knowledge proof system $\text{ChPeZKP} = (P_5, V_5, R_5)$

Disputes and Resolution.

Distributed Key Generation.

Guardian G_i part in Setup, with $i \in L$ *publish public keys of guardians $\forall j \in L$*

```

1 :  $(pk_i, sk_i) \leftarrow \text{DPKE.KGen}(pp)$ 
2 :  $\pi_i \leftarrow \text{SchZKP.P}(sk_i : pp, pk_i)$ 
3 :  $PL \leftarrow (i, pk_i, \pi_i)$ 
4 : wait and collect  $\forall j \neq i. (j, pk_j, \pi_j)$ 
5 :  $\forall j \neq i. \text{ if } \neg \text{SchZKP.V}(pp, pk_j, \pi_j) \text{ then}$ 
6 :    $evidence \leftarrow (j, pk_j, \pi_j)$ 
7 :    $O \leftarrow O \cup \{\text{Dispute}(i, j, evidence)\}$ 
8 :  $epk = \prod_{j \in L \setminus O} pk_j$ 

```

Exchange shares of guardian public keys

```

9 :  $((sh_{i,1}, \dots, sh_{i,n}), pub_i) \leftarrow \text{VSS-v2.0.0.Share}(pp, sk_i, [0x10, i])$ 
10 :  $caux \leftarrow (\text{label}, 0x00, \text{context}, 0x0200)$ 
11 :  $\forall j \neq i. (c_{i,j}, r_{i,j}) \leftarrow \text{EPKE.Enc}(pp, pk_j, sh_{i,j}, [0x11, i, j], [caux])$ 
12 :  $PL \leftarrow (pk_i, pub_i)$ 
13 : individual send  $c_{i,j}$  to Guardian  $G_j$ 
14 : wait and collect  $\forall j \neq i. (pk_j, pk_i, c_{j,i})$ 
15 :  $\forall j \neq i. sh_{j,i} \leftarrow \text{EPKE.Dec}(pp, sk_i, c_{j,i}, [0x11, j, i], [caux])$ 
16 :  $\forall j \neq i. \text{ if } \neg \text{VSS-v2.0.0.Vf}(pp, pub_j, sh_{j,i}, [0x10, j, i]) \text{ then}$ 
17 :    $evidence_1 \leftarrow (pk_i, pk_j, sh_{j,i}, c_{j,i})$ 
18 :    $evidence_2 \leftarrow (pk_i, pk_j, c_{j,i}, r_{j,i})$ 
19 :    $O \leftarrow O \cup \{\text{Dispute}(i, j, evidence)\}$ 
20 :  $esk_i \leftarrow \sum_{j=0}^n sh_{j,i} \bmod q$  notice that also shares of  $sk_i$  are used

```

computer public key of election

```

21 :  $epk = \prod_{j \in L \setminus O} pk_j$ 
22 :  $H_E \leftarrow H(H_B; 0x12, epk)$ 
23 : return  $(H_E, epk, esk_i)$ 

```

Vote (id, epk, v, [baux])

```

1 :  $rb \leftarrow \$\mathbb{Z}_{2^{256}}$ 
2 :  $r \leftarrow H(H_E; 0x20, rb)$ 
3 :  $cip \leftarrow \text{DPKE.Enc}(epk, v; r)$ 
4 :  $\pi \leftarrow \text{ChPeZKP.P}((v, r) : pp, cip, [0x21])$ 
5 :  $cc \leftarrow H(H_E; 0x24, epk, cip, [baux])$ 
6 :  $b \leftarrow (id, cip, \pi, cc, [baux])$ 
7 : return  $b$ 

```

Audit (pk, b, v, r)

```

1 :  $(id, cip, \pi, cc, [baux]) \leftarrow b$ 
2 :  $cip' \leftarrow \text{DPKE.Enc}(epk, v; r)$ 
3 :  $e_1 \leftarrow (cip = cip')$ 
4 :  $e_2 \leftarrow (cc = H(H_E; 0x24, pk, cip, [baux]))$ 
5 :  $e_3 \leftarrow \text{ChPeZKP.V}(pp, cip, \pi, [0x21])$ 
6 : return  $(e_1 \wedge e_2 \wedge e_3)$ 

```

Append (PBB, b)

```

1 : if  $\text{Valid}(PBB, b)$  then
2 :    $PBB \leftarrow PBB \cup \{b\}$ 
3 : return  $PBB$ 

```

Guardian G_i part in Tally, with $i \in J$ *compute partial result res_i*

```

1 : if  $\neg \text{Valid}(PBB, epk)$  then return  $\perp$ 
2 :  $(A, B) \leftarrow \text{DPKE.Add}(PBB)$ 
3 :  $A_i \leftarrow \text{DPKE.Dec}(esk_i, (A, B))$ 
4 :  $PL \leftarrow (i, A_i)$ 

```

compute result - anyone can publicly compute

```

5 : wait and collect  $\forall j. (j, A_j)$ 

```

Catalin : aux for VSS?

```

6 :  $(M, \{L_j \mid j \in J\}) \leftarrow \text{VSS-v2.0.0.RecE}(pp, J, \{A_i \mid i \in I\})$ 
7 :  $result \leftarrow B \cdot M^{-1} \bmod p$ 

```

prove partial results

```

8 :  $lesk_i = esk_i \cdot L_i$ 
9 :  $st \leftarrow (pp, \{(A_i, lepki) \mid i \in J\}, J, (A, B), [0x30])$ 
10 :  $\pi \leftarrow \text{MPZKP.P}(\{lesk_i \mid i \in J\} : st)$ 
11 : return  $(result, \pi)$ 

```

Valid (PBB, b)

```

1 :  $(id, cip, \pi, cc, [baux]) \leftarrow b$ 
2 :  $e_1 \leftarrow \text{ChPeZKP.Vf}(pp, cip, \pi, [0x21])$ 
3 :  $e_2 \leftarrow (cc = H(H_E; 0x24, epk, cip, [baux]))$ 
4 :  $e_3 \leftarrow \forall b' \in PBB. b' \neq (id, cip, \pi, cc, [baux])$ 
5 : return  $(e_1 \wedge e_2 \wedge e_3)$ 

```

Vf(pk, PBB, result, $\{(A_i, lepki) \mid i \in J\}$)

```

1 :  $st \leftarrow (pp, \{(A_i, lepki) \mid i \in J\}, J, (A, B), [0x30])$ 
2 :  $e \leftarrow \text{MPZKP.V}(st, \pi)$ 
3 : return  $e$ 

```

4. ElectionGuard v2.1.0

Catalin: to fix H and Hq calls

Entities and their role.

- **Election Authority** EA sets the election manifest that contains for example, the list of candidates, list of eligible voters, number of guardians and threshold.
- **Guardians** G own a share of the secret key, such that collectively they provide the election result and ensure correctness of the process while maintaining privacy. We consider a threshold model (t, n) where at least t out of the total n Guardians are needed.
- **Voters** id cast their vote.

Parameter generation and verification. Before the election starts, the following public parameters are agreed $pp = (\text{ver}, (p, q, g), \text{manifest}, t, n)$, where $\text{ver} = \text{"v2.1.0"}$, and $(p, q, g) \leftarrow \text{\$ GSetup}$ are a cyclic group and (t, n) the threshold and total number of guardians. The Election Authority defines $(H_P, H_B) \leftarrow \text{GenHash}(pp)$, and anyone can verify by recomputing the values.

GenHash(pp) with $pp = (\text{ver}, (p, q, g), \text{manifest}, t, n)$		
1 :	$H_P \leftarrow H(\text{ver}; 0x00, (p, q, g), n, t)$	/ Parameter Base Hash
2 :	$H_B \leftarrow H(H_M; 0x01, \text{manifest})$	/ Election Base Hash
3 :	return (H_P, H_B)	

4.1. Primitives

We consider the public key encryption schemes defined in Section 2.1: $\text{DPKE} = (\text{KGen}_1, \text{Enc}_1, \text{Dec}_1, \text{Add}_1)$, $\text{EPKE} = (\text{KGen}_2, \text{Enc}_2, \text{Dec}_2)$, the zero-knowledge proof system $\text{SchZKP} = (P_3, V_3, R_3)$, verifiable secret sharing scheme $\text{CLVSS}_4 = (\text{Share}_4, \text{Rec}_4, \text{Vf}_4)$, the zero-knowledge proof system $\text{ChPeZKP} = (P_5, V_5, R_5)$

Disputes and Resolution.

Distributed Key Generation.

Guardian G_i part in Setup, with $i \in L$

I publish public keys of guardians $\forall j \in L$

```

1 : (pki, ski) ← DPKE.KGen(pp)
2 : (compki, comski) ← DPKE.KGen(pp)
3 : πi ← SchZKP.P(ski : pp, pki, [compki])
4 : PL ← (i, pki, compki, πi)
5 : wait and collect  $\forall j \neq i. (j, pk_j, compk_j, \pi_j)$ 
6 :  $\forall j \neq i.$  if  $\neg \text{SchZKP.V}(\text{pp}, pk_j, \pi_j, [\text{compk}_j])$  then
7 :   evidence ← (j, pkj, compkj, πj)
8 :   O ← O ∪ {Dispute(i, j, evidence)}
9 : epk =  $\prod_{j \in L \setminus O} pk_j$ 

I Exchange shares of guardian public keys

10 : ((shi,1, ..., shi,n), pubi) ← VSS-v2.1.0.Share(pp, ski, [0x10, i])
11 : caux ← (label, 0x00, context, 0x0200)
12 :  $\forall j \neq i.$  (ci,j, ri,j) ← EPKE-v2.1.0.Enc(pp, compkj, shi,j, [0x11, i, j], [caux])
13 : PL ← (pki, pubi)
14 : individual send ci,j to Guardian  $G_j$ 
15 : wait and collect  $\forall j \neq i. (pk_j, pk_i, c_{j,i})$ 
16 :  $\forall j \neq i.$  shj,i ← EPKE.Dec(pp, ski, cj,i, [0x11, j, i], [caux])
17 :  $\forall j \neq i.$  if  $\neg \text{VSS-v2.1.0.Vf}(\text{pp}, \text{pub}_j, \text{sh}_{j,i}, [0x10, j, i])$  then
18 :   evidence1 ← (pki, pkj, shj,i, cj,i)
19 :   evidence2 ← (pki, pkj, cj,i, rj,i)
20 :   O ← O ∪ {Dispute(i, j, evidence)}
21 : eski ←  $\sum_{j=0}^n sh_{j,i} \bmod q$  I notice that also shares of ski are used

```

I computer public key of election

```

22 : epk =  $\prod_{j \in L \setminus O} pk_j$ 
23 : HE ← H(HB; 0x14, epk,  $\widehat{\text{epk}}$ )
Catalin: fix the calls to epk hats, including eski hat
24 : HG ← H(HE; 0x13, epk,  $\widehat{\text{epk}}$ , {pubj | j ∈ L}, { $\widehat{\text{pub}}_j$  | j ∈ L}, {compkj | j ∈ L})
25 : return ((HE, HG, epk,  $\widehat{\text{epk}}$ ), (eski,  $\widehat{\text{eski}}$ , comski))

```

Vote (id, epk, v, [baux])

```

1 : rb ←  $\$ \mathbb{Z}_{2^{256}}$ 
2 : r ← H(HE; 0x20, rb)
3 : cip ← DPKE.Enc(epk, v; r)
4 : π ← ChPeZKP.P((v, r) : pp, cip, [0x21])
5 : cc ← H(HE; 0x24, epk, cip, [baux])
6 : b ← (id, cip, π, cc, [baux])
7 : return b

```

Audit (pk, b, v, r)

```

1 : (id, cip, π, cc, [baux]) ← b
2 : cip' ← DPKE.Enc(epk, v; r)
3 : e1 ← (cip = cip')
4 : e2 ← (cc = H(HE; 0x24, pk, cip, [baux]))
5 : e3 ← ChPeZKP.V(pp, cip, π, [0x21])
6 : return (e1 ∧ e2 ∧ e3)

```

Append (PBB, b)

```

1 : if Valid(PBB, b) then
2 :   PBB ← PBB ∪ {b}
3 : return PBB

```

Guardian G_i part in Tally, with $i \in J$

I compute partial result res_i

```

1 : if  $\neg \text{Valid}(\text{PBB}, \text{epk})$  then return ⊥
2 : (A, B) ← DPKE.Add(PBB)
3 : Ai ← DPKE.Dec(eski, (A, B))
4 : PL ← (i, Ai)

```

I compute result - anyone can publicly compute

```

5 : wait and collect  $\forall j. (j, A_j)$ 
Catalin : aux for VSS?
6 : (M, {Lj | j ∈ J}) ← VSS-v2.0.0.RecE(pp, PL)
7 : result ← B · M-1 mod p

```

I prove partial results

```

8 : leski = eski · Li
9 : st ← (pp, {(Ai, lepki) | i ∈ J}, I, (A, B), [0x10])
10 : π ← MPZKP.P({leski | i ∈ J} : st)
11 : return (result, π)

```

Valid (PBB, b)

```

1 : (id, cip, π, cc, [baux]) ← b
2 : e1 ← ChPeZKP.Vf(pp, cip, π, [0x21])
3 : e2 ← (cc = H(HE; 0x24, epk, cip, [baux]))
4 : e3 ←  $\forall b' \in \text{PBB}. b' \neq (id, cip, \pi, cc, [baux])$ 
5 : return (e1 ∧ e2 ∧ e3)

```

Vf(pk, PBB, result, {(A_i, lepk_i) | i ∈ J})

```

1 : st ← (pp, {(Ai, lepki) | i ∈ J}, J, (A, B), [0x10])
2 : e ← MPZKP.V(st, π)
3 : return e

```

Figure 4. V2.1.0

References

- [1] M. Chase and A. Lysyanskaya, "On signatures of knowledge," Cryptology ePrint Archive, Paper 2006/184, 2006. [Online]. Available: <https://eprint.iacr.org/2006/184>
- [2] N. K. Nguyen, E. Theodorakis, and B. Warinschi, "Lifting standard model reductions to common setup assumptions," in *Public-Key Cryptography - PKC 2022 - 25th IACR International Conference on Practice and Theory of Public-Key Cryptography, Virtual Event, March 8-11, 2022, Proceedings, Part II*, ser. Lecture Notes in Computer Science, G. Hanaoka, J. Shikata, and Y. Watanabe, Eds., vol. 13178. Springer, 2022, pp. 130–160. [Online]. Available: https://doi.org/10.1007/978-3-030-97131-1_5
- [3] F. Hao, "Schnorr Non-interactive Zero-Knowledge Proof," Internet Engineering Task Force Documents - RFC 8235, 2017.
- [4] Y. Chen and Y. Lindell, "Feldman's verifiable secret sharing for a dishonest majority," *IACR Commun. Cryptol.*, vol. 1, no. 1, p. 16, 2024. [Online]. Available: <https://doi.org/10.62056/ak2isgvtw>
- [5] D. Bernhard and B. Warinschi, "Cryptographic voting - A gentle introduction," in *Foundations of Security Analysis and Design VII - FOSAD 2012/2013 Tutorial Lectures*, ser. Lecture Notes in Computer Science, A. Aldini, J. López, and F. Martinelli, Eds., vol. 8604. Springer, 2013, pp. 167–211. [Online]. Available: https://doi.org/10.1007/978-3-319-10082-1_7
- [6] M. Backes, A. Kate, and A. Patra, "Computational verifiable secret sharing revisited," in *Proceedings of Advances in Cryptology - ASIACRYPT 2011*, ser. LNCS, D. H. Lee and X. Wang, Eds., vol. 7073. Springer, 2011, pp. 590–609. [Online]. Available: https://doi.org/10.1007/978-3-642-25385-0_32
- [7] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Advances in Cryptology - CRYPTO '91, 11th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1991, Proceedings*, ser. Lecture Notes in Computer Science, J. Feigenbaum, Ed., vol. 576. Springer, 1991, pp. 129–140. [Online]. Available: https://doi.org/10.1007/3-540-46766-1_9
- [8] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979. [Online]. Available: <https://doi.org/10.1145/359168.359176>
- [9] P. Feldman, "A practical scheme for non-interactive verifiable secret sharing," in *28th Annual Symposium on Foundations of Computer Science, Los Angeles, California, USA, 27-29 October 1987*. IEEE Computer Society, 1987, pp. 427–437. [Online]. Available: <https://doi.org/10.1109/SFCS.1987.4>
- [10] J. C. Benaloh, "Secret sharing homomorphisms: Keeping shares of A secret sharing," in *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings*, ser. Lecture Notes in Computer Science, A. M. Odlyzko, Ed., vol. 263. Springer, 1986, pp. 251–260. [Online]. Available: https://doi.org/10.1007/3-540-47721-7_19
- [11] M. Ben-Or, S. Goldwasser, J. Kilian, and A. Wigderson, "Multi-prover interactive proofs: How to remove intractability assumptions," in *Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2-4, 1988, Chicago, Illinois, USA*, J. Simon, Ed. ACM, 1988, pp. 113–131. [Online]. Available: <https://doi.org/10.1145/62212.62223>
- [12] H. Cui, K. Zhang, Y. Chen, Z. Liu, and Y. Yu, "Mpc-in-multi-heads: A multi-prover zero-knowledge proof system - (or: How to jointly prove any NP statements in ZK)," in *Computer Security - ESORICS 2021 - 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4-8, 2021, Proceedings, Part II*, ser. Lecture Notes in Computer Science, E. Bertino, H. Schulmann, and M. Waidner, Eds., vol. 12973. Springer, 2021, pp. 332–351. [Online]. Available: https://doi.org/10.1007/978-3-030-88428-4_17
- [13] C. Asmuth and J. Bloom, "A modular approach to key safeguarding," *IEEE Trans. Inf. Theory*, vol. 29, no. 2, pp. 208–210, 1983. [Online]. Available: <https://doi.org/10.1109/TIT.1983.1056651>
- [14] M. Barzu, F. L. Tiplea, and C. C. Dragan, "Compact sequences of co-primes and their applications to the security of crt-based threshold schemes," *Inf. Sci.*, vol. 240, pp. 161–172, 2013. [Online]. Available: <https://doi.org/10.1016/j.ins.2013.03.062>
- [15] T. H. Yuen, H. Cui, and X. Xie, "Compact zero-knowledge proofs for threshold ECDSA with trustless setup," in *Public-Key Cryptography - PKC 2021 - 24th IACR International Conference on Practice and Theory of Public Key Cryptography, Virtual Event, May 10-13, 2021, Proceedings, Part I*, ser. Lecture Notes in Computer Science, J. A. Garay, Ed., vol. 12710. Springer, 2021, pp. 481–511. [Online]. Available: https://doi.org/10.1007/978-3-030-75245-3_18
- [16] J. Benaloh, M. Naehrig, O. Pereira, and D. S. Wallach, "Electionguard: a cryptographic toolkit to enable verifiable elections," in *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, D. Balzarotti and W. Xu, Eds. USENIX Association, 2024. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/benaloh>
- [17] D. Bernhard, "Zero-knowledge proofs in theory and practice," Ph.D. dissertation, University of Bristol, 2014.
- [18] V. Cortier, C. C. Dragan, F. Dupressoir, B. Schmidt, P. Strub, and B. Warinschi, "Machine-checked proofs of privacy for electronic voting protocols," in *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*. IEEE Computer Society, 2017, pp. 993–1008. [Online]. Available: <https://doi.org/10.1109/SP.2017.28>
- [19] M. Bellare, A. Desai, D. Pointcheval, and P. Rogaway, "Relations among notions of security for public-key encryption schemes," in *Advances in Cryptology - CRYPTO '98, 18th Annual International Cryptology Conference, Santa Barbara, California, USA, August 23-27, 1998, Proceedings*, ser. Lecture Notes in Computer Science, H. Krawczyk, Ed., vol. 1462. Springer, 1998, pp. 26–45. [Online]. Available: <https://doi.org/10.1007/BFb0055718>
- [20] D. Boneh, A. Partap, and L. Rotem, "Accountability for misbehavior in threshold decryption via threshold traitor tracing," in *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*, ser. Lecture Notes in Computer Science, L. Reyzin and D. Stebila, Eds., vol. 14926. Springer, 2024, pp. 317–351. [Online]. Available: https://doi.org/10.1007/978-3-031-68394-7_11
- [21] H. Devillez, O. Pereira, and T. Peters, "How to verifiably encrypt many bits for an election?" in *Computer Security - ESORICS 2022 - 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26-30, 2022, Proceedings, Part II*, ser. Lecture Notes in Computer Science, V. Atluri, R. D. Pietro, C. D. Jensen, and W. Meng, Eds., vol. 13555. Springer, 2022, pp. 653–671. [Online]. Available: https://doi.org/10.1007/978-3-031-17146-8_32
- [22] M. Abdalla, M. Bellare, and P. Rogaway, "The oracle diffie-hellman assumptions and an analysis of DHIES," in *Topics in Cryptology - CT-RSA 2001, The Cryptographer's Track at RSA Conference 2001, San Francisco, CA, USA, April 8-12, 2001, Proceedings*, ser. Lecture Notes in Computer Science, D. Naccache, Ed., vol. 2020. Springer, 2001, pp. 143–158. [Online]. Available: https://doi.org/10.1007/3-540-45353-9_12
- [23] M. Bellare and C. Namprepmpre, "Authenticated encryption: Relations among notions and analysis of the generic composition paradigm," in *Advances in Cryptology - ASIACRYPT 2000, 6th International Conference on the Theory and Application of Cryptology and Information Security, Kyoto, Japan, December 3-7, 2000, Proceedings*, ser. Lecture Notes in Computer Science, T. Okamoto, Ed., vol. 1976. Springer, 2000, pp. 531–545. [Online]. Available: https://doi.org/10.1007/3-540-44448-3_41
- [24] J. H. An, Y. Dodis, and T. Rabin, "On the Security of Joint Signature and Encryption," in *EUROCRYPT 2002*, ser. LNCS, vol. 2332, 2002, pp. 83–107.

Appendix A.

Additional Building Blocks Details

A.1. Threshold PKE Definition

The threshold encryption public key encryption [20] $TPKE(t, n) = (\text{KGen}, \text{Enc}, \text{Dec}, \text{Combine})$ for $0 < t \leq n$

- $(pk, pkc, sk_1, \dots, sk_n) \leftarrow \text{KGen}(\lambda, t, n)$ takes the security parameter λ and produces a public key pk , a number of secret keys $sk_1, \dots, sk_n$, and a combiner public key pkc -secret key pair.
- $c \leftarrow \text{Enc}(pk, m)$ returns the ciphertext c that encrypts the message m using the public key pk . When we need the randomness we use $c \leftarrow \text{Enc}(pk, m; r)$ or $(c, r) \leftarrow \text{Enc}(pk, m)$.
- $d_i \leftarrow \text{Dec}(sk_i, c)$ returns a partial decryption d_i using parties i secret key sk_i .
- $m_{\perp} \leftarrow \text{Combine}(pkc, c, I, \{d_i\}_{i \in I})$ returns either a valid message m or an error symbol $\perp$. It combines the partial decryptions of I users.

Appendix B.

Standard Instantiations for Zero-Knowledge

In Figure 5 we provide the pseudocode for the standard zero-knowledge instantiations for Schnorr proofs [3] (case $m=1$), Chaum-Pederson proofs [5], and proofs of multiple-exponentiations [4] (a generalized version of Schnorr).

Appendix C.

Proof for Lemma ??

Catalin: provide a proof for this.

ChPeZKP: Chaum-Pederson zero-knowledge proof $R((v, a), pp, (A, B))$ for $v \in \{0, 1\}$ 1 : $((p, q, g), H_p, pk) \leftarrow pp$ 2 : $e \leftarrow ((A = g^a \wedge B = pk^{a+v}) \vee (A = g^a \wedge B = pk^{a+(1-v)}))$ 3 : return e $P((v, a) : pp, (A, B)[aux])$ 1 : $((p, q, g), H_p, pk) \leftarrow pp$ 2 : $u_0, u_1, c_0, c_1 \leftarrow \mathbb{Z}_q$ 3 : $(A_0, B_0) \leftarrow (g^{u_0}, pk^{u_0+vc_0})$ 4 : $(A_1, B_1) \leftarrow (g^{u_1}, pk^{u_1+(v-1)c_1})$ 5 : $c \leftarrow H(H_E; [aux], pk, A, B, A_0, B_0, A_1, B_1)$ 6 : $c_v \leftarrow (c - c_{1-v}) \bmod q$ 7 : $z_0 \leftarrow (u_0 - c_0 \cdot a) \bmod q$ 8 : $z_1 \leftarrow (u_1 - c_1 \cdot a) \bmod q$ 9 : $\pi \leftarrow (c_0, c_1, z_0, z_1)$ 10 : return π $V((pp, (A, B)[aux]), \pi)$ 1 : $((p, q, g), H_p, pk) \leftarrow pp$ 2 : $(c_0, c_1, z_0, z_1) \leftarrow \pi$ 3 : $(A_0, B_0) \leftarrow (g^{z_0} A^{c_0}, pk^{z_0} B^{c_0})$ 4 : $(A_1, B_1) \leftarrow (g^{z_1} A^{c_1}, pk^{z_1-c_1} B^{c_1})$ 5 : $c' \leftarrow H(H_p; [aux], pk, A, B, A_0, B_0, A_1, B_1)$ 6 : return $(c = c')$	ExpoZKP: Exponential zero-knowledge proof $R((a_0, \dots, a_m) : pp, A_0, \dots, A_m)$ 1 : $((p, q, g), H_p) \leftarrow pp$ 2 : $\forall i \in \{0, \dots, m\}. e_i \leftarrow A_i = g^{a_i} \bmod p$ 3 : $e \leftarrow e_0 \wedge \dots \wedge e_m$ 4 : return e $P((a_0, \dots, a_m) : pp, A_0, \dots, A_m, [aux])$ 1 : $((p, q, g), H_p) \leftarrow pp$ 2 : $r_0, \dots, r_m \leftarrow \mathbb{Z}_q$ 3 : $\forall i \in \{0, \dots, m\}. h_i \leftarrow g^{r_i} \bmod p$ 4 : $c \leftarrow H_q(H_p; [aux], A_0, \dots, A_m, h_0, \dots, h_m)$ 5 : $\forall i \in \{0, \dots, m\}. z_i \leftarrow (r_i + c \cdot a_i) \bmod q$ 6 : $\pi \leftarrow (c, z_0, \dots, z_m)$ 7 : return π $V((pp, A_0, \dots, A_m, [aux]), \pi)$ 1 : $((p, q, g), H_p) \leftarrow pp$ 2 : $(c, z_0, \dots, z_m) \leftarrow \pi$ 3 : $\forall i \in \{0, \dots, m\}. h'_i \leftarrow (g^{z_i} / A_i^c) \bmod p$ 4 : $c' \leftarrow H_q(H_p; [aux], A_0, \dots, A_m, h'_0, \dots, h'_m)$ 5 : return $(c = c')$	ExpoZKP: Schnorr proof $R(a : pp, A)$ 1 : $((p, q, g), H_p) \leftarrow pp$ 2 : $e \leftarrow A = g^a \bmod p$ 3 : return e $P_3(a : pp, A, [aux])$ 1 : $((p, q, g), H_p) \leftarrow pp$ 2 : $r \leftarrow \mathbb{Z}_q$ 3 : $h \leftarrow g^r \bmod p$ 4 : $c \leftarrow H(H_p; [aux], A, h)$ 5 : $z \leftarrow (r + c \cdot a) \bmod q$ 6 : $\pi \leftarrow (c, z)$ 7 : return π $V_3((pp, A, [aux]), \pi)$ 1 : $((p, q, g), H_p, H_B, H_E) \leftarrow pp$ 2 : $(c, z) \leftarrow \pi$ 3 : $h \leftarrow (g^z / A^c) \bmod p$ 4 : $c' \leftarrow H(H_p; [aux], A, h)$ 5 : return $(c = c')$
---	---	--

Figure 5. Instantiations for Zero-Knowledge proof.